

VIETNAM NATIONAL UNIVERSITY HO CHI MINH
CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND
ENGINEERING



GRADUATION THESIS

THESIS

**BUILDING A FRAMEWORK TO
GENERATE AI MODELS FOR
HUMAN MOTION TRACKING
APPLICATIONS**

Major: Computer Science

SUPERVISOR(s): Dr. Le Trong Nhan

—o0o—

STUDENT: Nguyen Truong Minh Hoang

STUDENT'S ID: 2270757

HO CHI MINH CITY, DECEMBER 2025

VERIFICATION OF INSTRUCTOR

Signature confirmation

Declaration

We guarantee that this research is our own, conducted under the supervision and guidance of Dr. Le Trong Nhan. The result of our research is legitimate and has not been published in any forms prior to this. All materials used within this researched are collected by ourselves, by various sources and are appropriately listed in the references section. In addition, within this research, we also used the results of several other authors and organizations. They have all been aptly referenced. In any case of plagiarism, we stand by our actions and are to be responsible for it. Ho Chi Minh City University of Technology therefore are not responsible for any copyright infringements conducted within our research.

Acknowledgements

I would like to express my heartfelt gratitude to everyone who has contributed to the development and refinement of this thesis proposal.

First and foremost, I extend my deepest appreciation to my academic supervisor, Le Trong Nhan, for his invaluable guidance, encouragement, and constructive feedback throughout the conceptualization of this proposal. His expertise and insights have been instrumental in shaping the direction of my research.

I am also profoundly grateful to my institution, Ho Chi Minh City University of Technology, and the Department of Computer Science for providing a nurturing academic environment and the necessary resources to undertake this research.

Special thanks to my peers and colleagues who have shared their perspectives and offered thoughtful suggestions, enriching the scope of this study.

Finally, I am immensely thankful to my family and friends for their unwavering support, patience, and motivation as I delve into this challenging yet rewarding endeavor.

This research proposal is a reflection of collective support and dedication, and I am deeply thankful to all those who have contributed directly or indirectly to its realization.

Abstract

The advent of machine learning and edge computing has revolutionized how motion tracking applications are developed and deployed, enabling real-time, energy-efficient, and accurate systems for diverse use cases. This proposal outlines a research study aimed at building a framework to generate AI models tailored for motion tracking applications. The framework will enable users to easily collect motion data, train models, and evaluate their performance using tools such as Edge Impulse and Google MediaPipe.

Focusing on the Seeed XIAO nRF52840 Sense as the primary hardware platform, this research explores its capabilities for motion detection and BLE communication to smartphones. By leveraging this device's onboard IMU sensor and its compatibility with edge AI tools, the study aims to demonstrate a scalable and practical solution for motion tracking. The research also investigates the challenges of real-time motion tracking, such as data quality, model optimization, and integration with resource-constrained edge devices.

The proposed framework seeks to bridge the gap between complex machine learning workflows and non-expert users, empowering them to build and deploy motion tracking applications efficiently. The findings and methodologies presented in this study will lay the groundwork for the future development of motion tracking systems in areas such as healthcare, sports analytics, and robotics.

Contents

List of Tables

List of Figures

Chapter 1

Introduction

1.1 Background

Motion tracking is a dynamic and rapidly evolving field that involves the capture, processing, and analysis of motion, whether it is the movement of objects, individuals, or environments. According to^[?], the technology plays a pivotal role in a variety of fields. In healthcare, motion tracking supports physical rehabilitation, aids in fall detection, and aids in patient monitoring. In sports, it enhances performance tracking, aids in injury prevention, and provides insights to athletes and coaches. In gaming and virtual reality (VR) or augmented reality (AR), it promotes immersive and interactive user experiences. Furthermore, in robotics, motion tracking supports navigation systems and enables precise control mechanisms.

Advances in sensor technology and artificial intelligence (AI) have been transformative, dramatically increasing the accuracy, efficiency, and flexibility of motion tracking systems. In particular, AI models are indispensable for interpreting motion data, leading to applications such as gesture recognition, activity classification, and predictive motion modeling. However, the process of developing these AI models is inherently complex. It involves several complex steps, including collecting motion data, preprocessing and filtering that data to remove noise, extracting meaningful features, and finally training machine learning or deep learning models. This high level of complexity poses a significant challenge, especially for developers and researchers who may lack expertise in AI or motion tracking technology.

To overcome these challenges and expand the accessibility of motion tracking technologies, there is an increasing focus on creating frameworks designed to simplify the creation of AI models. These frameworks aim to democratize the field, making motion tracking and AI integration more accessible to a wider audience, including not

only seasoned experts and researchers, but also developers and enthusiasts with limited expertise. By providing streamlined tools and processes, such frameworks promise to lower barriers to entry and foster innovation.

1.2 Problem Statement

Despite advancements in motion tracking technologies, generating AI models for motion analysis remains a challenging task. Current approaches often require extensive technical knowledge, domain-specific expertise, and significant time and resources. Moreover, existing solutions are frequently tailored to specific applications, lacking the flexibility and scalability needed to adapt to diverse use cases.

For example, popular frameworks and tools often have steep learning curves, requiring users to invest significant effort to understand and utilize their capabilities. Additionally, these tools may offer limited functionality, limiting the ability to customize models for unique or emerging motion tracking requirements. Such limitations can stifle innovation, as they discourage experimentation and limit the adoption of AI-enabled motion tracking solutions in areas where they could have transformative impact.

There is therefore an urgent need for a flexible, intuitive, and user-friendly framework that simplifies the creation of AI models suitable for motion tracking applications. This framework must be designed to accommodate both experts and non-experts, empowering a diverse group of users to effectively exploit the potential of motion tracking technologies.

1.3 Research Objectives

The overarching goal of this study is to evaluate the feasibility and importance of developing a specific framework for generating AI models in the context of motion tracking. To achieve this, the study will address the following specific objectives:

- To thoroughly investigate the inherent challenges associated with designing and deploying AI models for motion tracking applications.
- To conduct a comprehensive analysis of existing frameworks and tools, such as Google’s Teachable Machine, to identify current limitations and unmet needs.
- To propose an initial conceptual design for a flexible, extensible, and user-friendly framework capable of supporting a wide range of motion tracking applications.

- To explore the potential benefits of such a framework in enabling more efficient and accessible model development processes.

This research aims to lay the foundation for a solution that not only simplifies the technical aspects of AI model creation, but also ensures that the resulting models maintain high standards of performance and adaptability.

1.4 Scope and Limitations

This research focuses on the intersection of artificial intelligence and motion tracking, with a particular emphasis on frameworks that facilitate the creation of AI models. The scope includes the following key areas:

- A detailed review of current methodologies, technologies, and tools used in motion tracking.
- An evaluation of sensor-based data collection techniques and pre-processing procedures to ensure data quality and reliability.
- Identification of opportunities to design a general-purpose AI model generator that addresses both common and unique challenges in motion tracking.
- Exploration of the technical design elements and user interfaces that would enable the widespread adoption of such a framework.

However, this research is subject to certain limitations. The focus will primarily be on high-level design and conceptual considerations rather than detailed implementation or coding. In addition, the study will not provide a comprehensive analysis of domain-specific applications, such as medical diagnostics or advanced sports analytics. Real-world testing may also be limited by hardware and computational constraints, which may impact the validation of proposed solutions.

Chapter 2

Literature Review

2.1 Motion Tracking Applications

Motion tracking, the process of capturing and analyzing the movements of objects or individuals, has become a cornerstone technology in various domains. The integration of advanced sensor systems and artificial intelligence (AI) has significantly enhanced its precision and applicability. Below is an overview of its key applications:

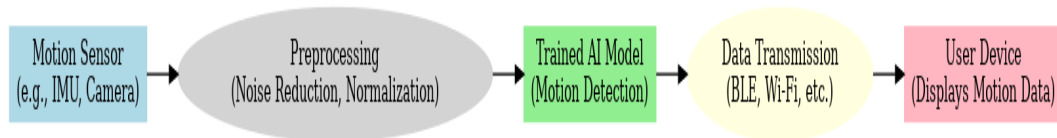


Figure 2.1.1: Motion Detection Process General

- **Healthcare**

Motion tracking technologies play a pivotal role in healthcare, particularly in physical rehabilitation and movement disorder diagnostics^[?]. Systems equipped with motion tracking capabilities allow healthcare professionals to monitor and evaluate patients' movements, ensuring rehabilitation exercises are performed with precision. Gait analysis, a specialized application, aids in diagnosing conditions such as Parkinson's disease or cerebral palsy, enabling tailored therapeutic interventions. Additionally, fall detection systems, leveraging real-time motion tracking, provide critical support in elder care, mitigating risks associated with sudden falls.

2.1. Motion Tracking Applications

Example: Wearable devices equipped with accelerometers and gyroscopes track gait patterns in patients with Parkinson's disease, enabling early detection of irregularities and tailored interventions.

Benefits: Enhances patient care, promotes early diagnosis, and provides data-driven insights for medical professionals.

- **Sports and Fitness**

In sports, motion tracking enhances performance monitoring and injury prevention. Athletes and coaches use motion analysis to optimize biomechanics, such as joint angles, posture, and movement efficiency. Wearable sensors and AI-based systems enable real-time feedback during training, helping athletes refine their techniques while reducing the likelihood of overuse injuries. Devices like smartwatches and fitness trackers analyze metrics such as stride length, cadence, and heart rate variability. Moreover, motion tracking facilitates data-driven approaches to performance improvement, fostering a competitive edge.

Example: Professional athletes use wearable motion trackers to refine their techniques and assess the biomechanics of their movements during training.

Advances: Real-time feedback mechanisms help coaches and athletes make instant adjustments, maximizing performance.

- **Gaming and Virtual/Augmented Reality (VR/AR)**

The gaming industry and immersive technologies like VR and AR heavily rely on motion tracking to provide intuitive and engaging user experiences. Full-body motion tracking systems, including motion controllers and VR suits, allow users to interact seamlessly with virtual environments. In AR, motion tracking aligns virtual objects with real-world movements, enabling applications in education, entertainment, and remote collaboration. These innovations contribute to the growing demand for lifelike and interactive digital experiences.

Example: Motion controllers in gaming consoles like the PlayStation VR track hand and body movements to provide realistic gameplay experiences.

Impact: Facilitates more natural user interactions and expands applications to virtual training and simulations.

- **Robotics and Automation**

Robotics applications depend on motion tracking for navigation, task execution, and human-robot interaction. Robots equipped with motion tracking systems can

perform complex tasks, such as autonomous navigation in dynamic environments or collaborative operations in industrial settings. For example, motion tracking enables precise path planning and obstacle avoidance, critical for autonomous delivery robots and robotic arms used in manufacturing.

Example: Robotic arms in manufacturing use motion tracking to precisely perform repetitive tasks like assembly and packaging.

Benefits: Boosts efficiency, accuracy, and safety in industrial applications.

- **Emerging Applications**

Beyond traditional domains, motion tracking is gaining traction in areas such as filmmaking, where it underpins motion capture (mocap) technology to create lifelike animations. In biomechanics, motion tracking supports research on human movement and ergonomics. Smart home systems utilize gesture-based motion tracking for intuitive control of devices, exemplifying its potential in consumer technologies.

Role of AI in Motion Tracking

Artificial intelligence has become indispensable in advancing motion tracking systems. By processing data from inertial measurement units (IMUs)^[?], cameras^[?], or other sensors, AI models enable accurate classification and prediction of movements. For instance, AI algorithms can distinguish between activities such as walking, running, or jumping using sensor data from wearable devices. The integration of AI not only enhances the precision of motion tracking but also democratizes its applications by automating complex analyses, thereby broadening accessibility across diverse user groups.

Table 2.1.1: Applications of Motion Tracking Across Domains

Application Domain	Examples	Key Benefits
Healthcare	Gait analysis, fall detection	Accurate monitoring, personalized treatment plans
Sports	Performance tracking, injury prevention	Optimized training, reduced injury risks
Gaming/VR/AR	Immersive VR/AR experiences	Enhanced user interaction, lifelike simulations
Robotics	Path planning, human-robot interaction	Autonomous navigation, precise task execution

2.2 AI Model Generation Techniques

The development of AI models for motion tracking involves several stages, from data collection to deployment. These models leverage sensor data, such as accelerometer and gyroscope readings, to detect, classify, and predict movements. Moreover, the generation of AI models for motion tracking involves various methodologies designed to extract meaningful patterns from raw sensor data. These techniques are tailored to meet the specific requirements of motion tracking applications, balancing accuracy, computational efficiency, and scalability. The following sections discuss the key techniques, tools, and frameworks for AI model generation in motion tracking applications.

2.2.1 Traditional Machine Learning Approaches

Traditional machine learning methods rely on extracting handcrafted features from motion data, such as mean, variance, and frequency-domain characteristics. These features are then used to train classifiers, such as Support Vector Machines (SVMs), Decision Trees, and k-Nearest Neighbors (kNN). While effective for specific applications, these methods often require significant domain expertise to design feature extraction pipelines and may struggle to generalize across diverse datasets.

2.2.2 Deep Learning Approaches

Deep learning has revolutionized motion tracking by automating feature extraction and achieving higher accuracy in complex tasks. Neural network architectures such as:

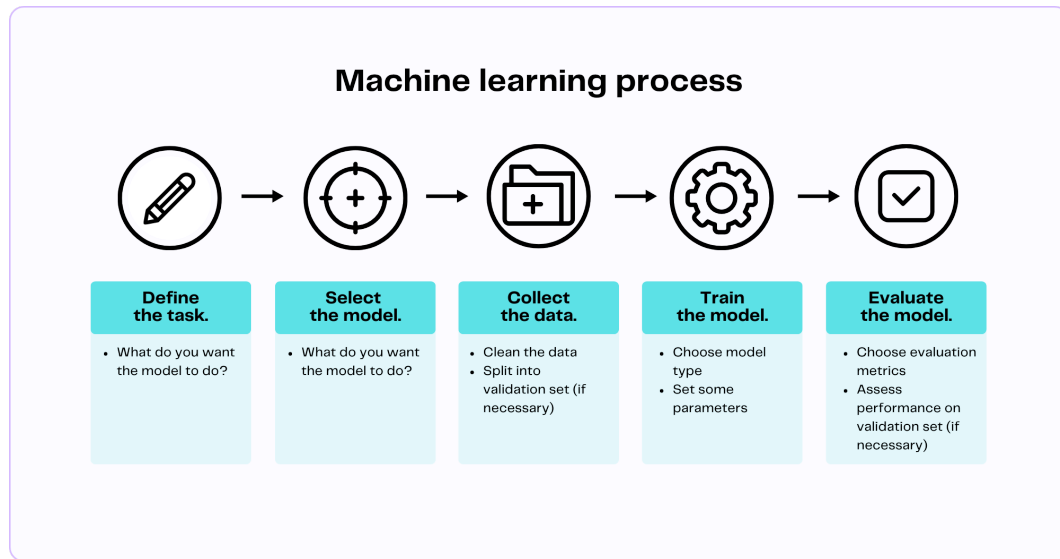


Figure 2.2.1: Machine learning process

- Convolutional Neural Networks (CNNs): Often used for processing spatially structured data like images or time-series motion data.
- Recurrent Neural Networks (RNNs) and Long Short-Term Memory Networks (LSTMs): Well-suited for handling sequential data, capturing temporal dependencies in motion signals.
- Transformer Models: Emerging as powerful tools for sequence modeling, with applications in activity recognition.

Deep learning models can directly process raw sensor data, eliminating the need for manual feature engineering. However, they often require large labeled datasets and computational resources, posing challenges for deployment on resource-constrained devices.

2.2.3 Hybrid Approaches

Hybrid techniques combine the strengths of traditional and deep learning approaches. For instance, traditional feature extraction methods may be integrated with neural networks to enhance model interpretability and efficiency. Such approaches are particularly useful when datasets are small or when computational resources are limited.

2.2.4 Tools and Frameworks for Model Generation

Several tools and frameworks facilitate the development of AI models for motion tracking. These include both general-purpose machine learning libraries and specialized tools designed for edge and real-time applications:

Table 2.2.1: AI Model Generation Techniques

Framework	Key Features	Best Suited For
Edge Impulse	AutoML, TinyML optimization	Resource-constrained devices
Google MediaPipe	Real-time tracking, multimodal pipelines	VR/AR and interactive applications
TensorFlow/PyTorch	Extensive libraries, flexibility	Custom AI model development

- **Edge Impulse**^[?]: A platform designed specifically for edge AI development. Edge Impulse allows developers to collect sensor data, preprocess it, and train lightweight machine learning models optimized for deployment on microcontrollers and other edge devices. Its integrated pipeline simplifies motion tracking by offering pre-built modules for sensor data ingestion and model training.
- **Google MediaPipe**^[?]: A powerful framework for building perception pipelines. MediaPipe provides state-of-the-art solutions for real-time motion tracking using pre-trained AI models, including hand and body pose estimation. It offers a modular design, enabling developers to integrate motion tracking capabilities into applications with minimal effort. MediaPipe’s optimization for mobile and edge devices makes it highly suitable for real-time motion analysis.
- **Google’s Teachable Machine**^[?]: A web-based tool that simplifies AI model creation for non-experts by providing a no-code interface for training classification models. While primarily focused on general classification tasks, it offers potential for motion tracking applications by enabling quick prototyping of sensor-based models.
- **TensorFlow Lite and PyTorch Mobile**^[?]: These frameworks provide tools for deploying lightweight AI models on edge devices. They support quantization and other optimizations, enabling real-time inference on resource-constrained hardware.

2.3. Benefits in Motion Tracking AI Models

Table 2.2.2: Tools and Frameworks Evaluation

Tool/Framework	Purpose	Ease of Use	Optimized for Edge?	Real-Time Capabilities
Edge Impulse	Sensor-based model development	High	Yes	Moderate
Google MediaPipe	Perception pipelines for motion	Moderate	Yes	High
TensorFlow Lite	Model deployment on edge devices	Moderate	Yes	Moderate

2.2.5 Data Requirements and Preprocessing

AI model generation heavily relies on high-quality datasets. In motion tracking, this involves capturing data from sensors like accelerometers, gyroscopes, or cameras. Preprocessing steps include:

- Noise Reduction: Filtering out artifacts caused by sensor drift or environmental factors.
- Normalization: Ensuring consistent scale across data features.
- Segmentation and Labeling: Dividing motion signals into meaningful segments for supervised learning.

The pre-processing phase is critical for model performance, as poorly processed data can lead to suboptimal results, regardless of the model used.

2.3 Benefits in Motion Tracking AI Models

Motion tracking AI models offer benefits across a wide range of domains, leveraging a combination of advanced algorithms and sensor technology to improve accuracy, efficiency, and user experience. Here are some of the key benefits:

- **Improved accuracy:** AI models process large volumes of sensor data to detect and classify motion with high accuracy, minimizing errors in applications such as healthcare diagnostics and sports performance analysis.
- **Real-time feedback:** Real-time processing enables immediate feedback, which is important for applications such as rehabilitation exercises, sports training, and interactive VR/AR systems.
- **Scalability:** AI models can be deployed across multiple platforms, from mobile devices to industrial robots, ensuring adaptability to a wide range of hardware configurations and use cases.
- **Personalization:** Machine learning algorithms enable motion tracking systems to be tailored to each user, improving the suitability and effectiveness of applications such as fitness tracking and rehabilitation.
- **Cost-effectiveness:** Automating complex tasks reduces the reliance on manual intervention, leading to cost savings in areas such as manufacturing and elder care.
- **Wider Accessibility:** Lightweight AI models optimized for edge devices make advanced motion tracking accessible in resource-constrained environments, expanding usage to underserved areas or lower-budget projects.
- **Data-Driven Insights:** AI-powered analytics generate actionable insights from motion data, supporting decision-making in sports coaching, healthcare treatment planning, and industrial operations.

By integrating these benefits, AI-driven motion tracking continues to transform traditional operations, enabling innovative solutions and empowering users across multiple domains.

2.4 Challenges in Motion Tracking AI Models

Despite significant advancements, the development of AI models for motion tracking faces several challenges. Addressing these obstacles is crucial for achieving robust, scalable, and real-time systems, particularly in resource-constrained environments.

1. Data Collection and Quality

2.4. Challenges in Motion Tracking AI Models

- **Sensor Noise:** Motion tracking heavily relies on data from IMUs, cameras, or other sensors. However, these devices often produce noisy signals due to environmental factors, hardware limitations, or sensor drift.
- **Dataset Diversity:** Capturing diverse datasets that account for variations in users, activities, and environments remains challenging. Models trained on narrow datasets often fail to generalize to unseen scenarios.
- **Labeling Complexity:** Accurate labeling of motion data is time-intensive and often requires domain expertise. Ambiguous or inconsistent labels can degrade model performance.

2. Computational Constraints

- **Edge Deployment:** Many motion tracking applications require AI models to run on edge devices like microcontrollers or smartphones. These devices have limited computational resources, making it difficult to deploy complex models without optimization.
- **Real-Time Processing:** Motion tracking systems often demand low-latency inference for real-time applications such as sports performance monitoring or interactive VR. Balancing model complexity and speed remains a significant challenge.

3. Generalizability and Adaptability

- **Domain-Specific Customization:** AI models for motion tracking must adapt to different use cases, such as healthcare, sports, or gaming. Creating adaptable models without significant retraining is a complex task.
- **Cross-Device Consistency:** Variations in sensor quality and configurations across devices often lead to inconsistencies in data, requiring robust model designs or calibration techniques.

4. Data Privacy and Security

- **Sensitive Data:** Motion tracking often involves personal data, such as healthcare metrics or identifiable movement patterns. Ensuring data privacy while enabling AI training and inference is a significant concern.
- **Federated Learning:** Emerging techniques like federated learning offer potential solutions by training models locally on devices without sharing raw data. However, these approaches introduce additional complexity and may reduce model performance.

5. Integration Challenges

- **Hardware-Software Compatibility:** Motion tracking systems require seamless integration between hardware (e.g., sensors) and software (e.g., AI frameworks). Discrepancies can lead to performance bottlenecks.
- **Scalability:** Scaling solutions across diverse hardware ecosystems and applications is difficult, particularly when designing for global or multi-user scenarios.

2.5 Challenges in Motion Tracking and AI Integration

Despite advancements, several challenges remain in the integration of AI and motion tracking technologies. Addressing these challenges is essential for widespread adoption and innovation.

1. Hardware Constraints

- **Resource Limitations:** Many motion tracking applications rely on low-power devices with limited processing and storage capabilities.
- **Impact:** AI models need to be optimized for efficient inference on edge devices without compromising accuracy or be trade-offs between model accuracy and computational efficiency.
- **Example:** Difficulty in running deep learning models on microcontrollers due to memory constraints.

2. Data Quality and Variability

- **Challenges:** Sensor data can be noisy, inconsistent, or affected by environmental factors such as temperature, humidity, or interference.
- **Impact:** Requires robust preprocessing techniques and models capable of handling variability.
- **Example:** Wearables giving inaccurate step counts due to motion artifacts.

3. Scalability of AI Models

- **Issue:** Training large-scale AI models for motion tracking requires extensive computational resources and large datasets.

2.6. Current Trends in Motion Tracking and AI Integration

- Impact: Limits the ability of small organizations or individual researchers to develop and deploy advanced systems.
- Solution: Use of pre-trained models and transfer learning.

4. Latency in Real-Time Applications

- Problem: Real-time applications like gaming, VR/AR, and robotics demand low-latency processing, which is challenging for computationally intensive AI models.
- Impact: Can lead to lag or poor user experience.
- Solution: Development of lightweight, optimized AI models using quantization or pruning.

5. Ethical and Privacy Concerns

- Concern: Collecting and processing motion data raises questions about user privacy and data security.
- Impact: Limits user adoption and raises regulatory challenges.
- Solution: Adoption of privacy-preserving techniques like federated learning and on-device processing.

2.6 Current Trends in Motion Tracking and AI Integration

In recent years, motion tracking has evolved significantly due to advancements in AI and sensor technology. Emerging trends highlight how these technologies are converging to deliver innovative solutions across various domains.

2.6.1 Wearable Motion Tracking Systems

Wearable devices equipped with IMUs, GPS, and other sensors have become mainstream in sports, healthcare, and consumer electronics. These systems leverage AI for real-time analysis and provide actionable insights.

- Advances in Wearables: AI has made these systems smarter by providing real-time insights and personalized feedback. Integration of AI models on low-power edge devices, enabling on-device analysis and feedback.

2.6. Current Trends in Motion Tracking and AI Integration

- Example: Smart fitness bands like Fitbit and Apple Watch use AI to analyze fall detection in elderly care, step counts, heart rates, and sleep patterns accurately.
- Impact: Improved accessibility and affordability of motion tracking systems for consumers.

2.6.2 AI-Driven Real-Time Tracking

Real-time motion tracking is critical in domains like gaming, VR/AR, and autonomous robotics. AI algorithms now support faster and more accurate motion prediction.

- Key Trend: Development of lightweight AI models that operate efficiently on edge devices, ensuring low latency in processing.
- Example: Google MediaPipe's real-time hand tracking achieves high accuracy with minimal computational requirements.
- Applications: Immersive VR/AR systems, autonomous robotics, and interactive gaming experiences.

2.6.3 Multimodal Sensor Fusion

Combining data from multiple sensors improves the robustness and accuracy of motion tracking systems. AI plays a crucial role in fusing data streams from IMUs, cameras, and LiDAR sensors.

- Example: Autonomous vehicles use multimodal sensor fusion for precise navigation in dynamic environments.
- Benefits: Enhanced motion prediction, reduced errors, and improved adaptability to environmental changes.

2.6.4 Edge AI and Federated Learning

The rise of edge computing has transformed how motion tracking data is processed.

- Edge AI: AI models are deployed on devices rather than relying on cloud infrastructure, reducing latency and improving privacy.

Example: Smart glasses with embedded AI for gesture recognition.

2.6. Current Trends in Motion Tracking and AI Integration

- Federated Learning: A privacy-preserving technique where AI models are trained locally on devices, and only model updates are shared with central servers.

Example: Smartphones using federated learning for activity recognition without sharing raw motion data.

2.6.5 Emerging Domains and Applications

AI-driven motion tracking is extending into less conventional areas, creating novel opportunities:

- Smart Cities: Monitoring pedestrian and traffic movements to optimize urban planning.
- Healthcare Innovations: Using motion data for predictive health diagnostics.
- Cinematic Applications: AI-assisted motion capture for creating realistic animations in films and video games.

Chapter 3

Experimental Setup and Results

3.1 Experimental Purpose

In order to test/experience and analyze pros and cons of the existing frameworks, this experiment will demonstrate the experimental setup and results derived from testing the capabilities of Edge Impulse for implementing motion tracking on the Seeed XIAO nRF52840 Sense. By establishing and testing a hardware-software pipeline, this experiment validates the feasibility of edge AI-based motion detection, addressing critical aspects of the problem analysis chapter, including resource constraints and real-time inference needs.

3.2 Hardware Configuration

3.2.1 Seeed XIAO nRF52840 Sense

The Seeed Studio XIAO nRF52840 Sense is a powerful, compact, and versatile development board designed for IoT (Internet of Things) applications and sensor-based projects. It integrates a wide range of capabilities, including Bluetooth Low Energy (BLE), a suite of environmental sensors, and an ultra-compact form factor. This makes it an excellent choice for both professional developers and hobbyists working on wearable devices, remote sensing, and wireless communication systems.^[?]

Features:

- Powerful wireless capabilities: Bluetooth 5.0 with onboard antenna
- Powerful CPU: Nordic nRF52840, ARM® Cortex®-M4 32-bit processor with FPU, 64 MHz



Figure 3.2.1: Seeed XIAO nRF52840 Sense

- Ultra-Low Power: Standby power consumption is less than $5\mu\text{A}$
- Battery charging chip: Supports lithium battery charge and discharge management
- Onboard 2 MB flash
- Onboard PDM microphone (only in Seeed Studio XIAO nRF52840 Sense)
- Onboard 6-axis LSM6DS3TR-C IMU (only in Seeed Studio XIAO nRF52840 Sense)
- Ultra Small Size: 21 x 17.8mm, Seeed Studio XIAO series classic form-factor for wearable devices
- Rich interfaces: 1xUART, 1xI2C, 1xSPI, 1xNFC, 1xSWD, 11xGPIO(PWM), 6xADC in XIAO nRF52840 (Sense); and 2xUART, 1xI2C, 2xSPI, 1xI2S, 1xNFC, 1xSWD, 18xGPIO(PWM), 6xADC in XIAO nRF52840 (Sense) Plus
- Single-sided components, surface mounting design

The built-in IMU sensor details: LSM6DS3:

- Accelerometer Range: $\pm 2\text{g}$ to $\pm 16\text{g}$.
- Gyroscope Range: $\pm 125\text{ dps}$ to $\pm 2000\text{ dps}$.
- Sampling Frequency: Configured at 50 Hz to balance accuracy and power consumption.

Bluetooth Low Energy (BLE):

3.2. Hardware Configuration

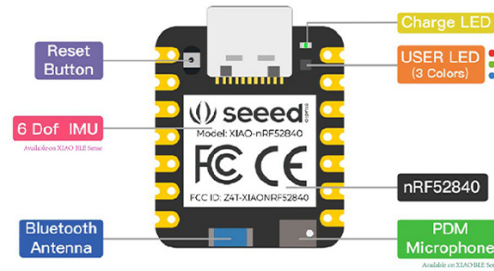


Figure 3.2.2: XIAO nRF52840 Sense front indication

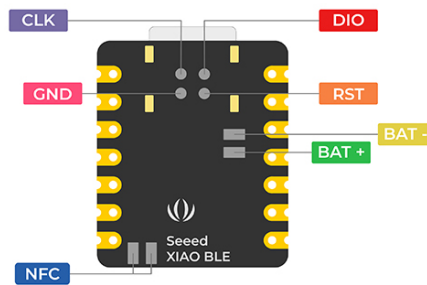


Figure 3.2.3: XIAO nRF52840 Sense back indication

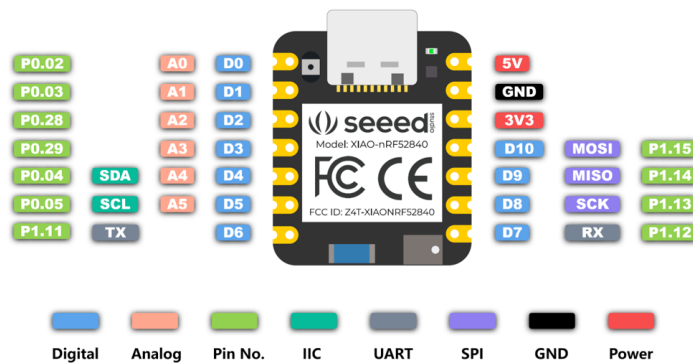


Figure 3.2.4: XIAO nRF52840 Sense Pin List

- Configured to send motion predictions in real-time to a paired smartphone.
- Achieved stable connectivity within a range of 10 meters indoors.

3.2.2 Setting up

We need to prepare the following:

- 1 x Seeed XIAO nRF52840 Sense
- 1 x Battery: 3.3V 80mAh

- 1 x Device housing with strap
- 1 x Computer
- 1 x USB Type-C cable

Connect the Seeed Studio XIAO nRF52840 (Sense) to your computer via a USB Type-C cable.



Figure 3.2.5: Connect the Seeed Studio XIAO nRF52840 Sense to computer via a USB Type-C cable.

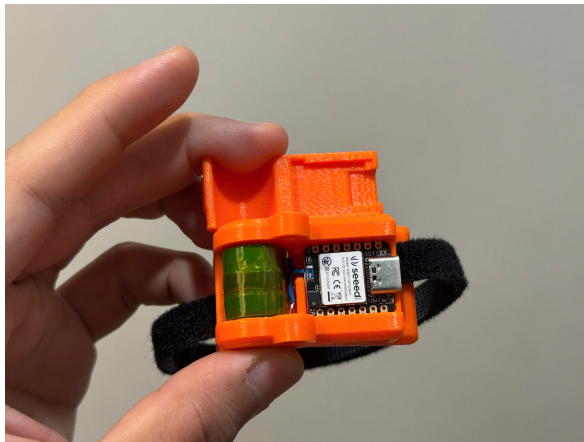


Figure 3.2.6: XIAO nRF52840 with battery

3.3 Software Configuration

3.3.1 Edge Impulse Studio

Edge Impulse Studio served as the main platform for the AI workflow:

1. Data Acquisition: Collected raw IMU data from the XIAO.

2. Signal Processing: Extracted relevant motion features using built-in spectral analysis tools.
3. Model Training: Automated training pipeline with options for real-time accuracy evaluation.
4. Deployment: Converted trained models to optimized TinyML formats compatible with the XIAO.

There is a draw back when directly capturing IMU data using Edge Impulse Studio is that it cannot recognize the gyroscope parameters from the built-in LSM6DS3. Hence, the experiment will use the UART communication to read the serial data from the board.

3.3.2 Arduino IDE

Used for firmware programming to enable:

- Sensor data streaming via BLE.
- Integration with Edge Impulse firmware for model inference.

To setup the working environment, the preparation should follow the below steps:

- Step 1. Download and Install the latest version of Arduino IDE according to your operating system.
- Step 2. Launch the Arduino application
- Step 3. Add Seeed Studio XIAO nRF52840 (Sense) board package to your Arduino IDE Navigate to File >Preferences, and fill "Additional Boards Manager URLs" with the url: https://files.seeedstudio.com/arduino/package_seeedduino_boards_index.json

Navigate to Tools >Board >Boards Manager..., type the keyword "seeed nrf52" in the search box, select the latest version of the board you want, and install it. You can install both.

It is recommended to use the **Seeed nRF52 mbed-enabled Boards** library if you want to use it in embedded Machine Learning Applications or apply "IMU and PDM advanced function".

- Step 4. Download and install the supported library from Seeed Studio to work with the built-in IMU easily: [Seeed Arduino LSM6DS3](#).

3.4 Data Acquisition

3.4.1 Data collection

1. Sensor Placement

Device affixed to the ankle using a strap for stability and consistent positioning.



Figure 3.4.1: Device affixed to ankle

2. Recording

Each activity was recorded for around 30 seconds using Arduino sketch to capture serial data. The sampling frequency using is 100Hz. One notice that the IMU address of the device is **0x6A**

```
// XIAO BLE Sense LSM6DS3 Accelerometer Raw Data

#include "LSM6DS3.h"

// Create a instance of class LSM6DS3
LSM6DS3 myIMU(I2C_MODE, 0x6A); // I2C device address 0x6A

#define CONVERT_G_TO_MS2 9.80665f
#define FREQUENCY_HZ 100
#define INTERVAL_MS (1000 / (FREQUENCY_HZ + 1))

static unsigned long last_interval_ms = 0;

void setup() {
```

```

Serial.begin(115200);
while (!Serial)
    ;

if (myIMU.begin() != 0) {
    Serial.println("Device_error");
} else {
    Serial.println("aX,aY,aZ,gX,gY,gZ");
}

void loop() {
    if (millis() > last_interval_ms + INTERVAL_MS) {
        last_interval_ms = millis();
        Serial.print(myIMU.readFloatAccelX() * CONVERT_G_TO_MS2
            ↪ , 4);
        Serial.print(',');
        Serial.print(myIMU.readFloatAccelY() * CONVERT_G_TO_MS2
            ↪ , 4);
        Serial.print(',');
        Serial.print(myIMU.readFloatAccelZ() * CONVERT_G_TO_MS2
            ↪ , 4);
        Serial.print(',');
        Serial.print(myIMU.readFloatGyroX() * CONVERT_G_TO_MS2,
            ↪ 4);
        Serial.print(',');
        Serial.print(myIMU.readFloatGyroY() * CONVERT_G_TO_MS2,
            ↪ 4);
        Serial.print(',');
        Serial.println(myIMU.readFloatGyroZ() *
            ↪ CONVERT_G_TO_MS2, 4);
    }
}

```

3. Labeling

Manual labeling during data acquisition to map samples to their respective ac-

tivities.

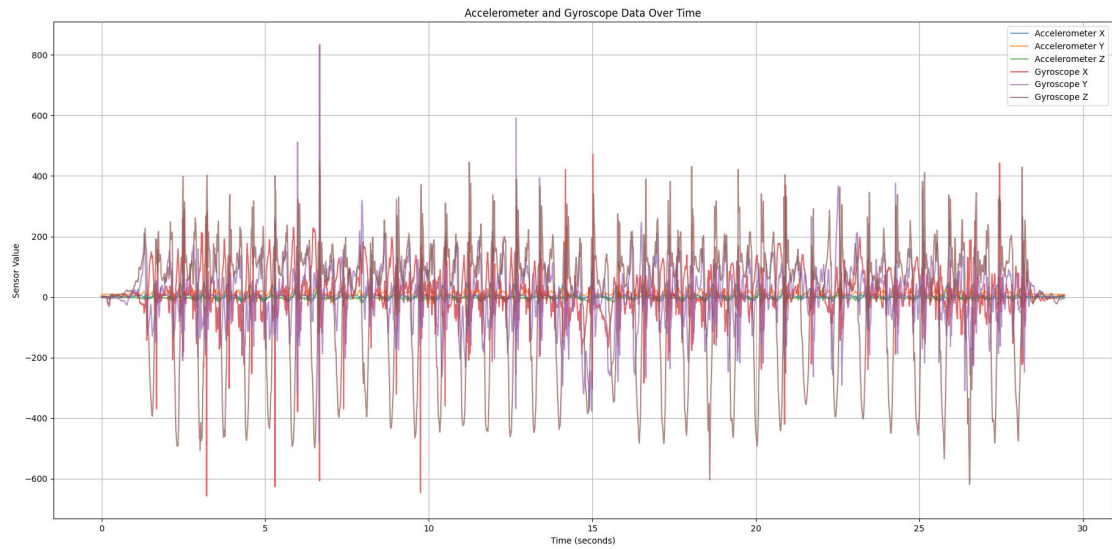


Figure 3.4.2: Raw data (100Hz): Running

3.4. Data Acquisition

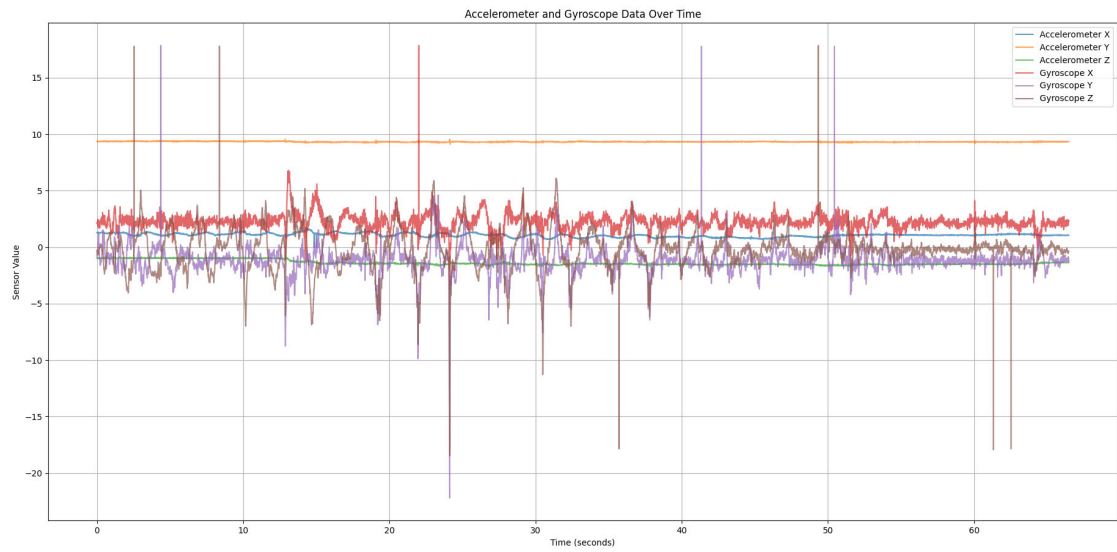


Figure 3.4.3: Raw data (100Hz): Standing

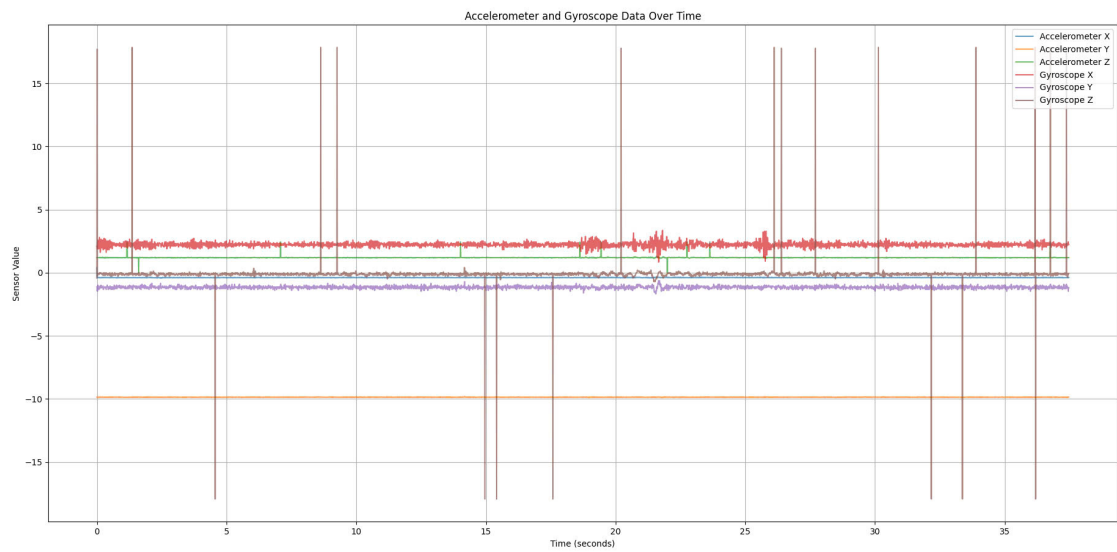


Figure 3.4.4: Raw data (100Hz): Not wearing

3.4. Data Acquisition

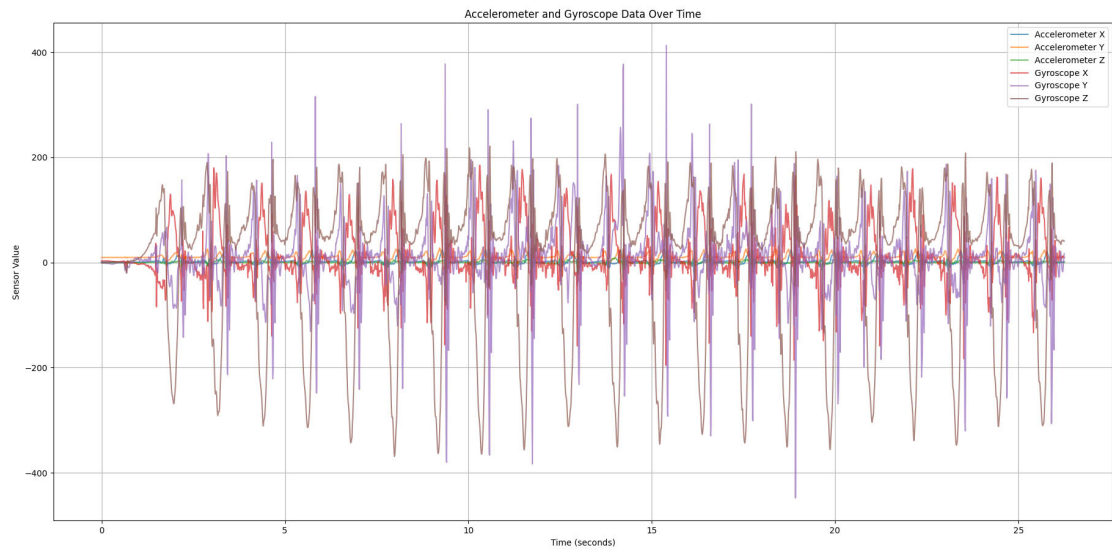


Figure 3.4.5: Raw data (100Hz): Walking

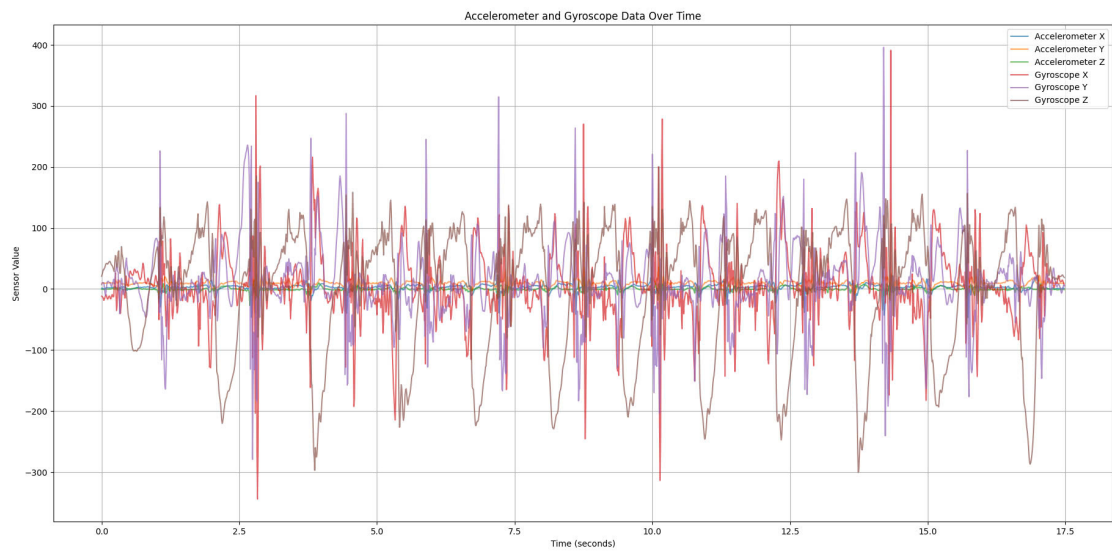


Figure 3.4.6: Raw data (100Hz): Walking downstairs

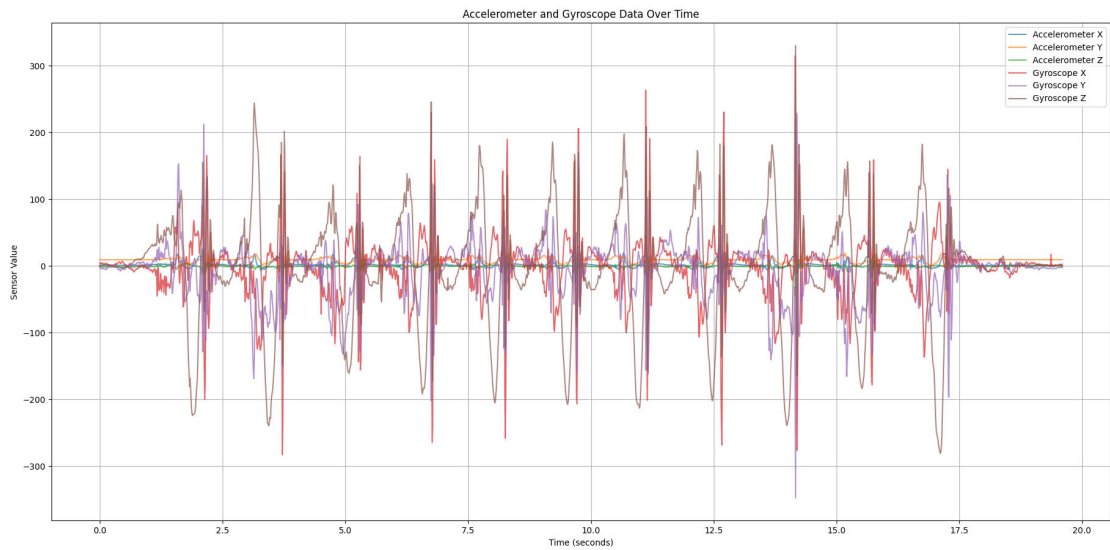


Figure 3.4.7: Raw data (100Hz): Walking upstairs

3.4.2 Data preprocessing

As the raw data are inconsistent and having many outliers that could lead to the ineffective training results. Therefore, we need to preprocess the data by cleaning and smoothening. To achieve that, Python is a great tool for such job.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import butter, filtfilt

# Load the dataset
filename = 'walking_upstairs'
file_path = f'{filename}.csv' # Replace with the correct file path
data = pd.read_csv(file_path)

# Specify the IMU columns
imu_columns = ['aX', 'aY', 'aZ', 'gX', 'gY', 'gZ']

# Function to visualize raw data
def visualize_data(data, title="IMU_Data"):
    plt.figure(figsize=(16, 8))
```

```

    for i, col in enumerate(data.columns):
        plt.subplot(2, 3, i + 1)
        plt.plot(data[col], label=col, linewidth=0.8)
        plt.title(f"{col}__{title}")
        plt.xlabel("Samples")
        plt.ylabel("Value")
        plt.legend()
        plt.grid()
    plt.tight_layout()
    plt.show()

# Visualize raw data
visualize_data(data[imu_columns], title="Raw_Data")

# Detect and remove outliers using Z-Score
def remove_outliers(data, threshold=3):
    z_scores = np.abs((data - data.mean()) / data.std())
    return data[(z_scores < threshold).all(axis=1)]

# Apply outlier removal
data_cleaned = remove_outliers(data[imu_columns])

# Function to apply a low-pass filter
def low_pass_filter(data, cutoff=5, fs=50, order=2):
    nyquist = 0.5 * fs
    normal_cutoff = cutoff / nyquist
    b, a = butter(order, normal_cutoff, btype='low', analog=False)
    return pd.DataFrame(filtfilt(b, a, data, axis=0), columns=data.
        ↪ columns)

# Apply low-pass filter
data_smoothed = low_pass_filter(data_cleaned, cutoff=5, fs=50)

# Visualize cleaned and smoothed data
visualize_data(data_cleaned, title="Cleaned_Data")
visualize_data(data_smoothed, title="Smoothed_Data")

```

```
# Save cleaned and smoothed data
output_cleaned_path = f'{filename}_cleaned.csv'
output_smoothed_path = f'{filename}_smoothed.csv'
data_cleaned.to_csv(output_cleaned_path, index=False)
data_smoothed.to_csv(output_smoothed_path, index=False)

print(f"Cleaned_data_saved_to_{output_cleaned_path}")
print(f"Smoothed_data_saved_to_{output_smoothed_path}")
```

After running the preprocessing job on the raw data, we can get the much better data samples.

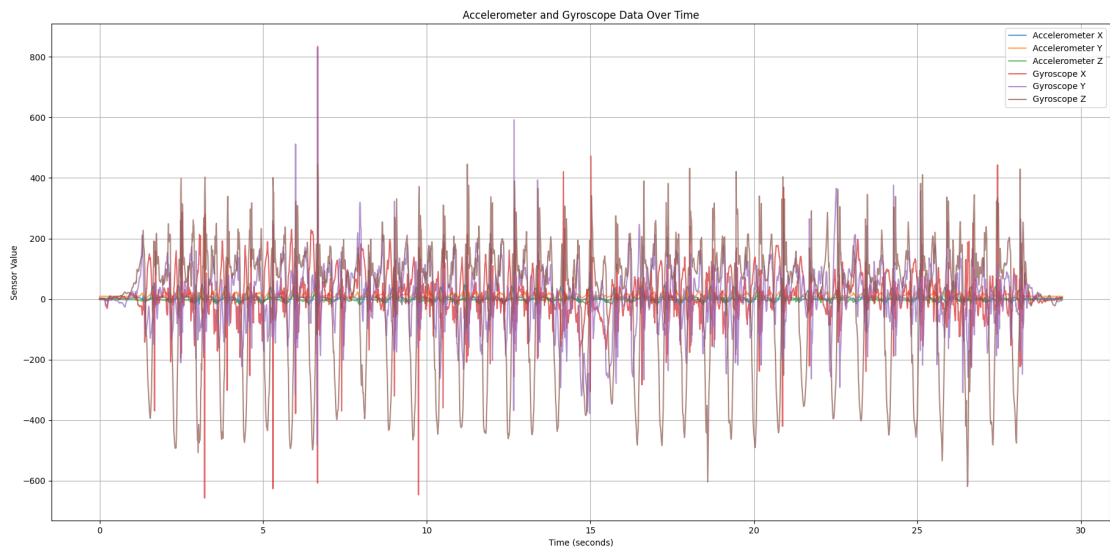


Figure 3.4.8: Preprocessed: Running

3.4. Data Acquisition

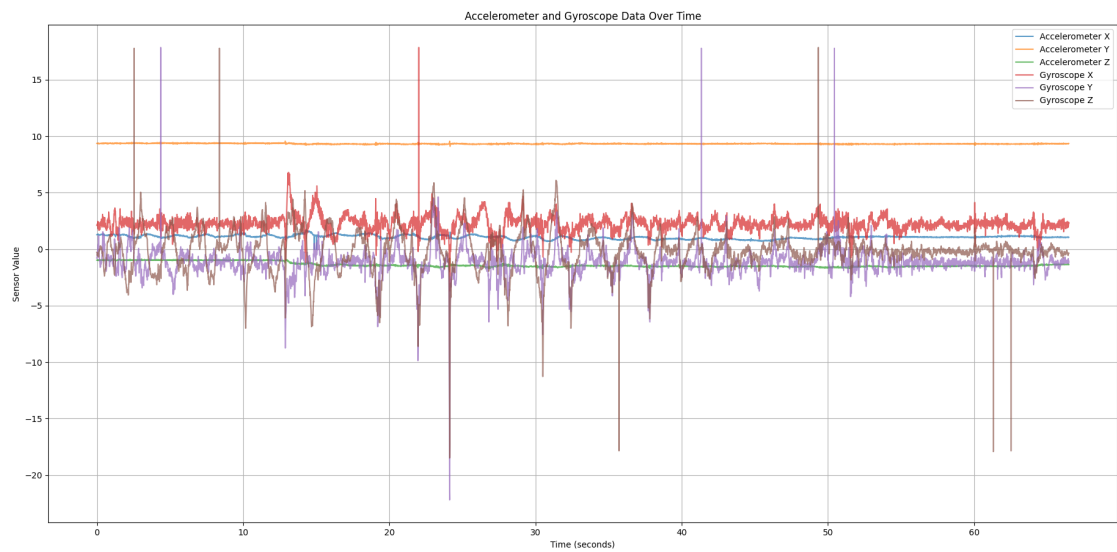


Figure 3.4.9: Preprocessed: Standing

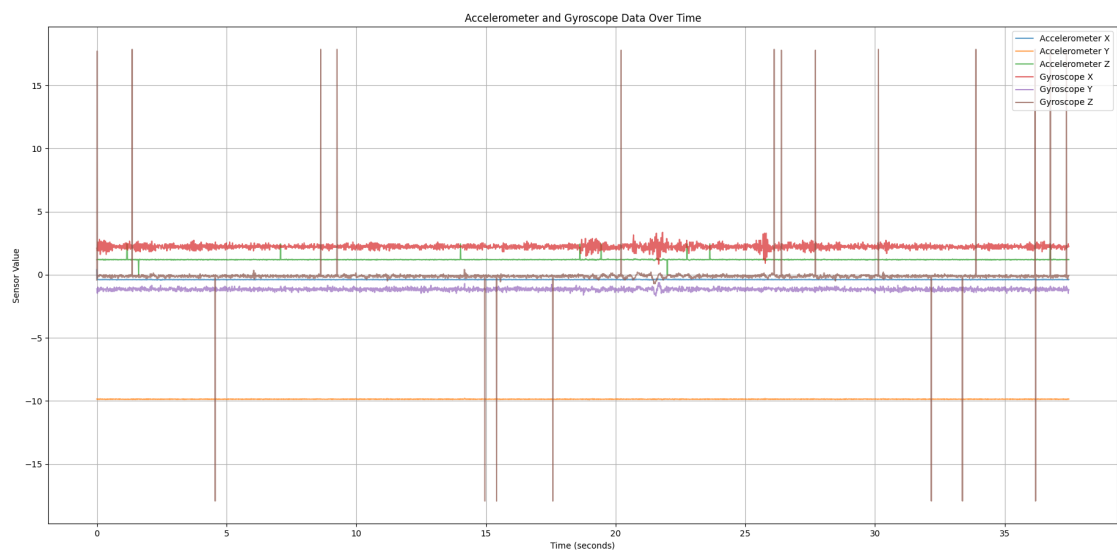


Figure 3.4.10: Preprocessed: Not wearing

3.4. Data Acquisition

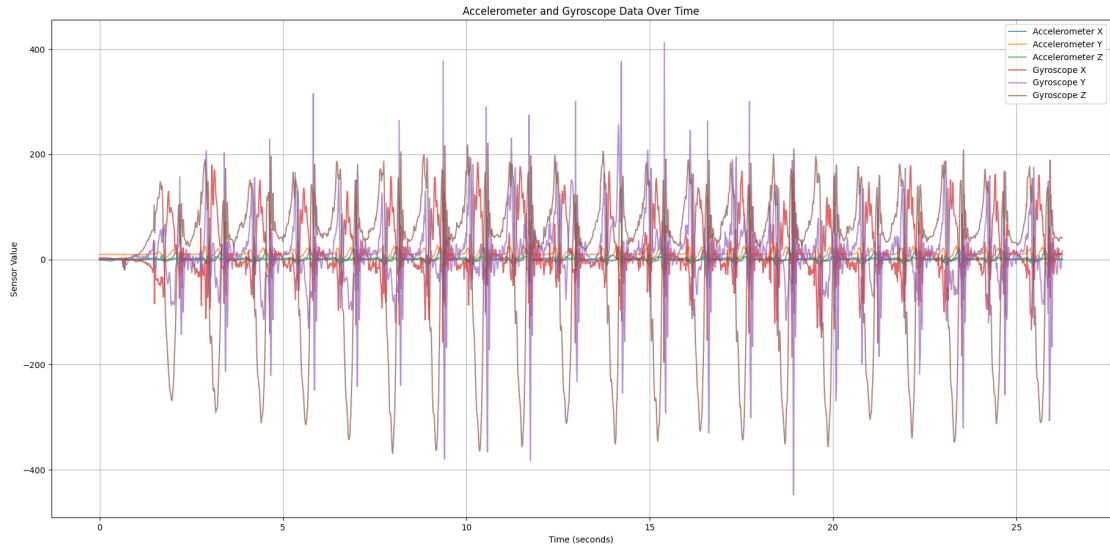


Figure 3.4.11: Preprocessed: Walking

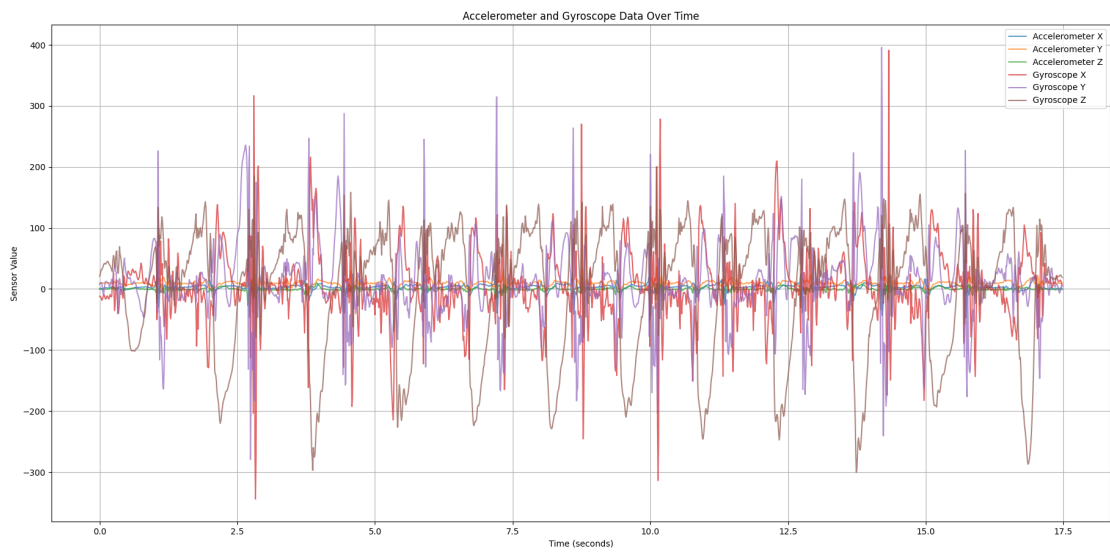


Figure 3.4.12: Preprocessed: Walking downstairs

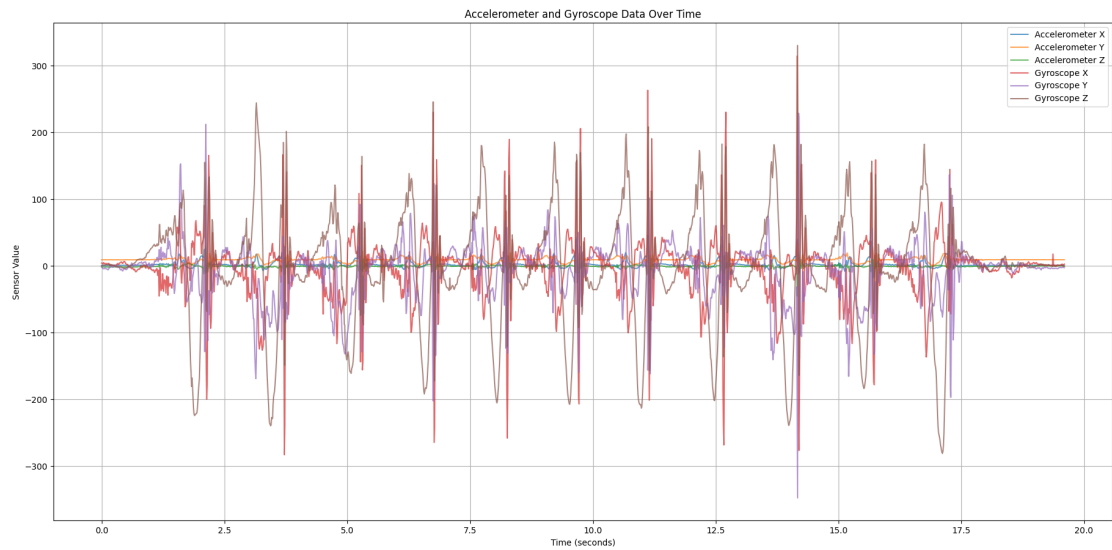


Figure 3.4.13: Preprocessed: Walking upstairs

3.5 Model Development

3.5.1 Dataset preparation

After having the preprocessed data, the next step is to upload the data samples to Edge Impulse project.

3.5. Model Development

Upload data

You can upload CBOR, JSON, CSV, WAV, JPG, PNG, AVI or MP4 files. You can also upload an annotation file named "info.labels" with your data to assign bounding boxes, labels, and/or metadata. View [Uploader docs](#) to learn more. Alternatively, you can use our [Python SDK](#) to programmatically ingest data in various formats, such as pandas or numpy.

CSV files will be processed using the active CSV Wizard configuration, which was created using "still_smoothed.csv" (Nov 18 2024, 00:41:12). [Download config](#).

Upload mode

☒ Select individual files ?

☐ Select a folder ?

Select files

running_smoothed.csv

Upload into category

☒ Automatically split between training and testing ?

☐ Training

☐ Testing

Label

☒ Infer from filename ?

☐ Leave data unlabeled ?

☐ Enter label:

Enter a label

< Back

Upload data

Figure 3.5.1: Edge Impulse Upload data

After uploading the data, Edge Impulse has a preview panel showing how our data look like. Moreover, it allows us to crop and split the data into multiple small samples with the label from the uploaded data.

By choosing split sample, Edge Impulse will pop up a window for us to adjust the splitting job. We can adjust the segment length to wider or narrower, or add more segments. After finish, the tool will break our data file into the number of segments that we specify in the split tool.

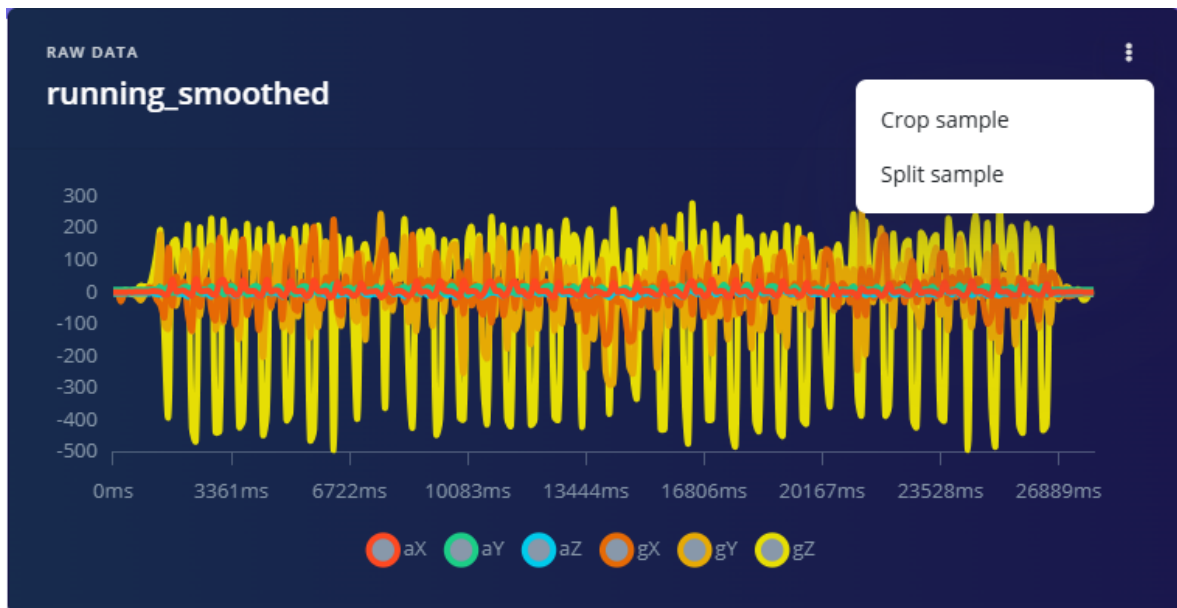


Figure 3.5.2: Edge Impulse Uploaded data preview

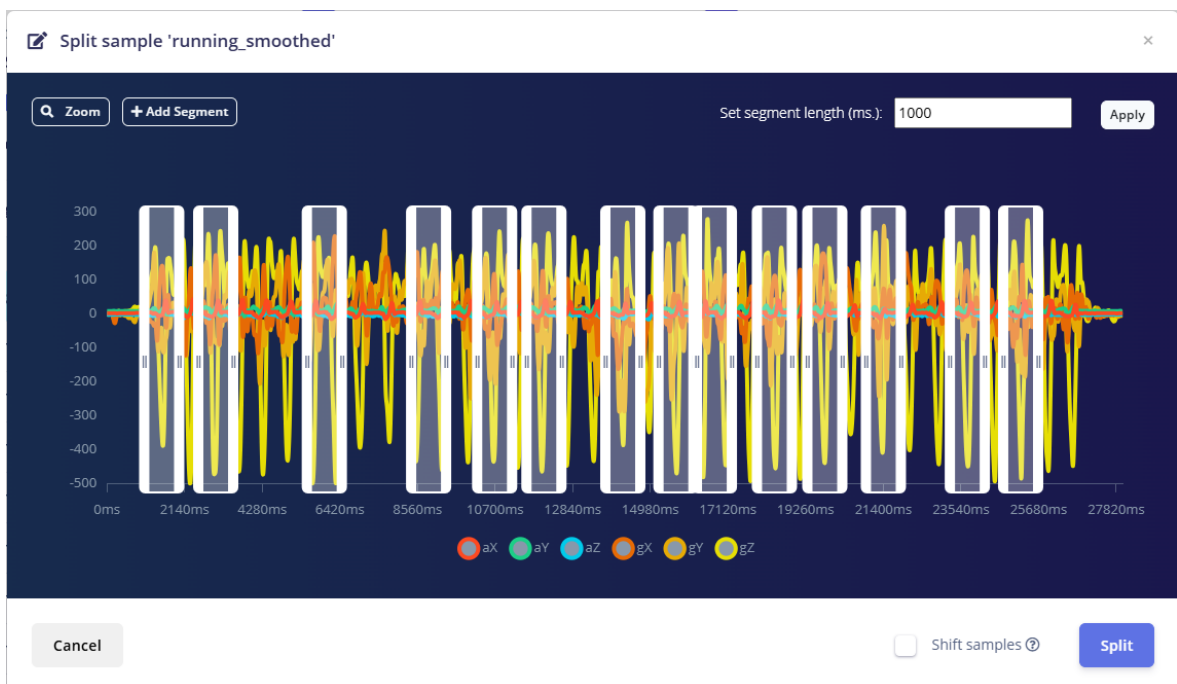


Figure 3.5.3: Edge Impulse Split sample

After splitting the data into samples, we can finally perform the train/test split to split samples into training dataset and testing dataset.

3.5.2 Impulse creation

1. Create impulse

3.5. Model Development

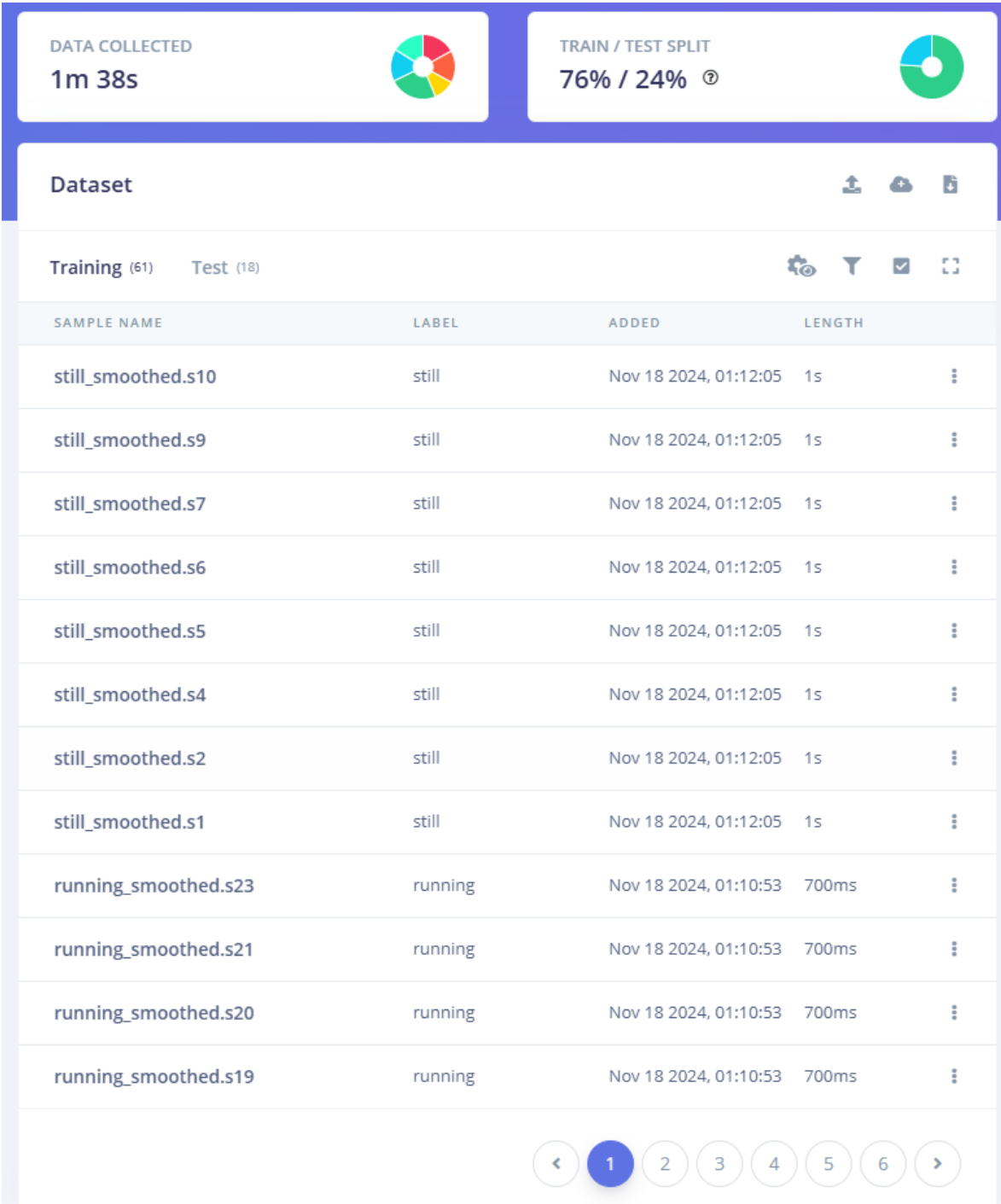


Figure 3.5.4: Edge Impulse Final dataset

First, we need to create an impulse which includes the data specifications block, processing block and learning block.

For the processing block, it is suggested to use the Spectral Analysis because it is great for analyzing repetitive motion, such as data from accelerometers and it

3.5. Model Development

extracts the frequency and power characteristics of a signal over time.

For the learning block, using Classification (Keras) is recommended. It learns patterns from data, and can apply these to new data and great for categorizing movement or recognizing audio.

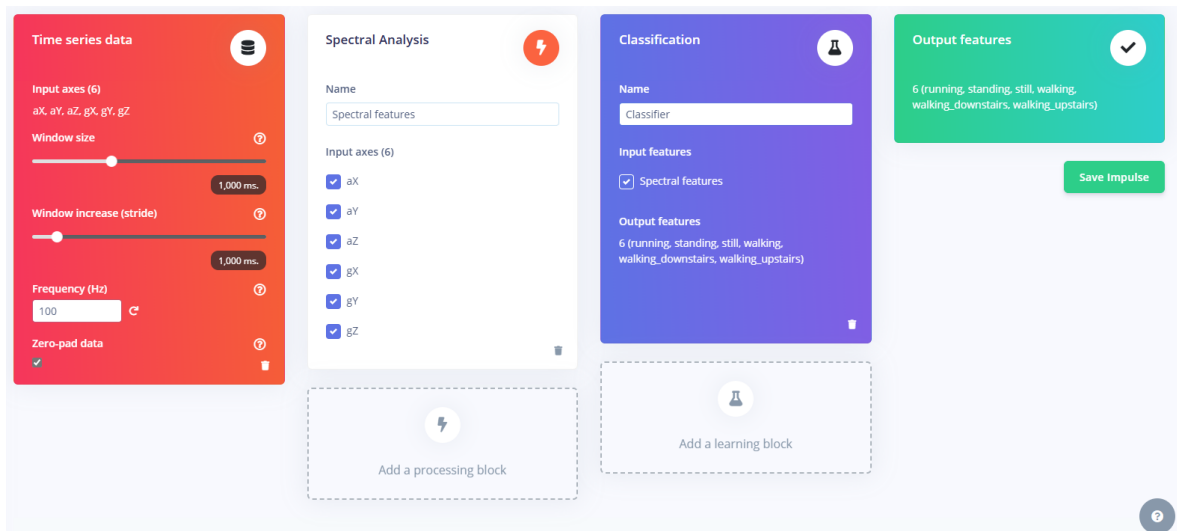


Figure 3.5.5: Create Impulse

2. Processing block

Then, by moving to the chosen processing block, we are able to generate features. In this case, the processing block is Spectral features. There are many parameters for us to determine.

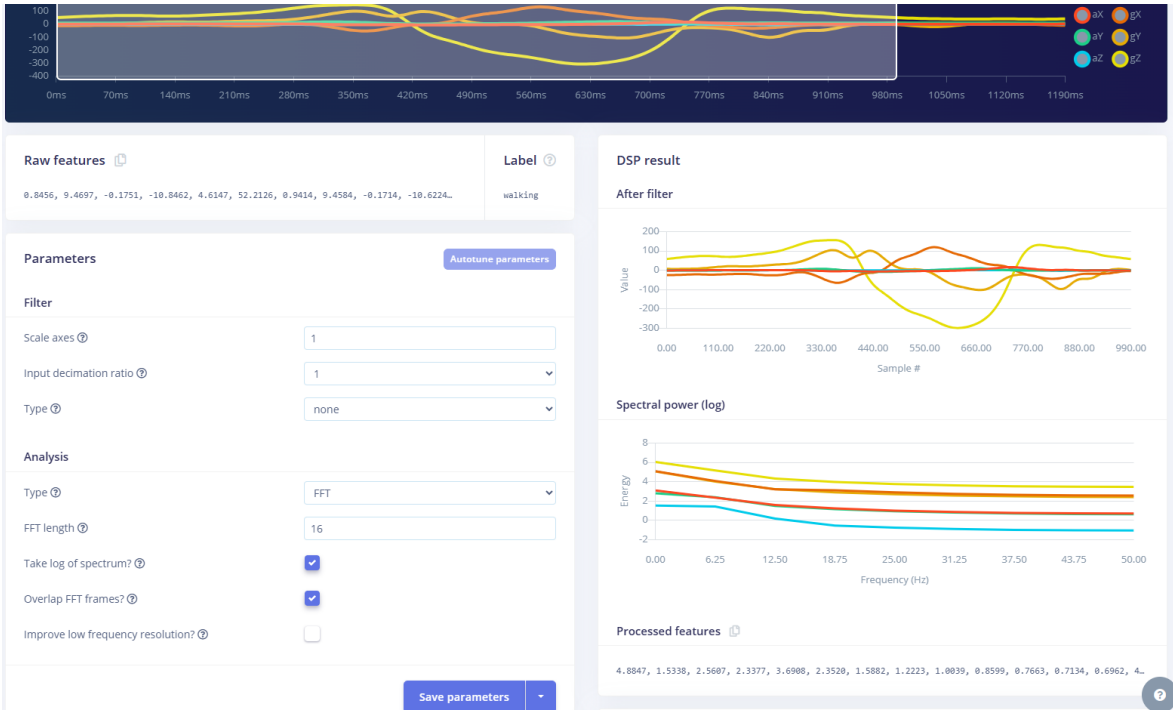


Figure 3.5.6: Spectral features parameters

Once we are satisfied with the parameters, clicking "Save parameters" will direct us to the next panel to generate the features. In this panel, it shows the overall training set information.

3.5. Model Development

By accepting to generate features, the page will start feature generation job. The output will show the spectrum of the differences between features.

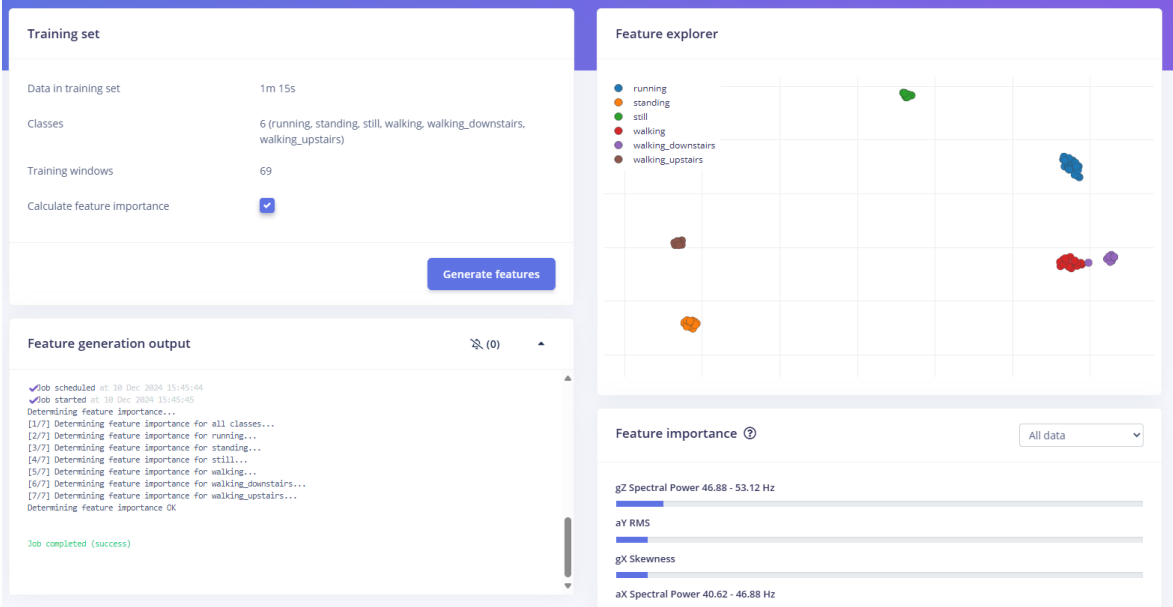


Figure 3.5.7: Generated features

As we can see in ??, most of the features are clearly separated, but the walking feature and walking downstairs feature are near to each other. This will lead to the inaccurate test results between these two.

3. Learning block

Coming to the learning block, we can set the parameters for the neural network settings.

Training settings

Number of training cycles ?

300

Use learned optimizer ?

☐

Learning rate ?

0.0005

Training processor ?

CPU

Advanced training settings

Validation set size ?

20

%

Split train/validation set on metadata key ?

Batch size ?

32

Auto-weight classes ?

☐

Profile int8 model ?

☒

Neural network architecture

Input layer (78 features)

Dense layer (20 neurons)

Dense layer (10 neurons)

Add an extra layer

Output layer (6 classes)

Figure 3.5.8: Neural network settings

3.5. Model Development

After setting the suitable parameters for this part, we can choose "Save and train" for it to start training job. The result of this step will be the confusion matrix, showing that the validation result of the trained model.



Figure 3.5.9: Validation result of trained model

3.5. Model Development

4. Model testing

Under the tab for model testing, it is allowed to do the testing on the newly trained model with the testing dataset that we prepared at the beginning.

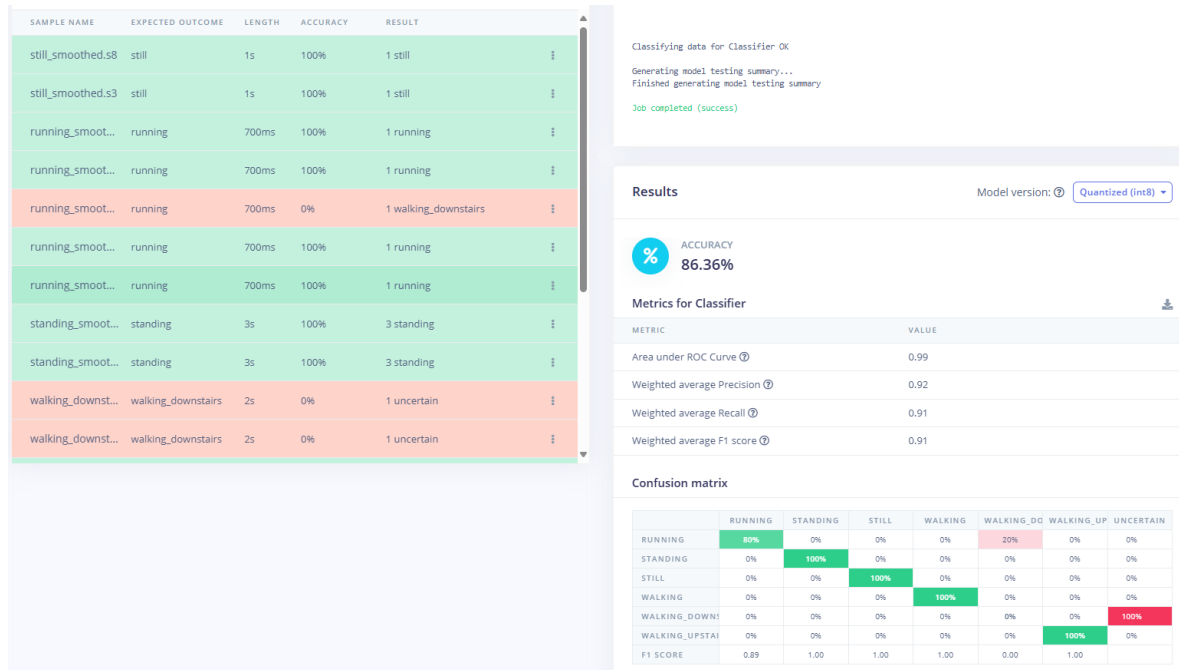


Figure 3.5.10: Model testing output

As the result, the accuracy is 86.36%, which is not ideal. The result shows that 100% of testing results on walking downstairs feature are uncertain.

5. Deployment

There are various options for deployment target. In this case, we will use the "Arduino library" and the target device is "Nordic nRF52840 DK (Cortex-M4F 64MHz)". After building, the page will allow us to download the generated model. In order to import the generated model to Arduino project, we can use "Add .ZIP library" option in Arduino IDE.

After importing the model, we can include the header file in the Arduino motion prediction project.

3.6 Results

3.6.1 Experiment results

After compiling a simple application to predict motions with the same setup for the ankle-mount device and flashing to the device, it is able to predict some types of motion but not all.

```
Starting inferencing in 2 seconds...
Sampling...
Predictions (DSP: 91 ms., Classification: 0 ms., Anomaly: 0 ms.):
    running: 0.05859
    standing: 0.00000
    still: 0.00000
    walking: 0.00000
    walking_downstairs: 0.25781
    walking_upstairs: 0.68359
    Best match: walking_upstairs

Starting inferencing in 2 seconds...
Sampling...
Predictions (DSP: 91 ms., Classification: 0 ms., Anomaly: 0 ms.):
    running: 0.04297
    standing: 0.00000
    still: 0.00000
    walking: 0.00000
    walking_downstairs: 0.02344
    walking_upstairs: 0.93359
    Best match: walking_upstairs
```

Figure 3.6.1: Motion detection results

In ??, the logs are showing the matching percentage for each activity and the best match for the prediction.

The results are telling that the "walking" motion is not detected, and mistaken to "walking_downstairs" or "walking_upstairs". Otherwise, the actual motions "walking_downstairs" and "walking_upstairs" are usually correct, but not always. Finally, the motions as "running", "standing" or "still" are highly accurate.

The inaccurate result is foreseen in the ??. Reasons for this can be from the source of the data, the preprocessing process, the data split methods, or anything that is in the way to create the complete application. Then, we can see the way to improve this is to minimize the possible errors can be caused during the process to produce the final

application.

3.6.2 Edge Impulse assessments

Pros:

- **Ease of Use:** User-friendly interface and streamlined workflow for building machine learning models without needing deep expertise in data science.
- **Optimized for Edge Devices:** Designed to optimize models for edge devices with limited resources, ensuring efficient deployment.
- **Wide Hardware Support:** Compatible with a variety of edge devices and sensors, making it versatile for many IoT applications.
- **Automated Model Optimization:** Simplifies model training and optimization for resource-constrained environments, reducing manual effort.
- **Cloud-Based:** Allows for easy collaboration and access to tools from anywhere with an internet connection.
- **Free Tier Available:** Offers a free version with useful features for smaller projects or testing.
- **Built-In Data Collection Tools:** Includes built-in tools for collecting and labeling data directly from sensors, reducing setup time.
- **Deployment Flexibility:** Supports deployment to multiple platforms (e.g., micro-controllers, mobile devices) with minimal setup.

Cons:

- **Limited Customization:** Less flexibility for advanced users who need fine-grained control over model architecture or training techniques.
- **Dependency on Internet:** Requires a stable internet connection for training and deployment, which may not be ideal for all environments.
- **Resource Limitations for Complex Models:** Performance may suffer when deploying large or computationally demanding models due to the resource constraints of edge devices.

- **Challenges with Niche Hardware:** Integrating with custom or non-standard hardware may require additional work or custom development.
- **Paid Features for Scale:** Advanced features and scaling capabilities require a paid subscription, which can become expensive for large projects.
- **Data Privacy Concerns:** Cloud-based nature could raise privacy or compliance concerns, particularly for sensitive data.
- **Learning Curve for Beginners:** While user-friendly, beginners without machine learning experience may still face challenges in data collection, labeling, and model creation.
- **Integration Issues:** Integrating Edge Impulse with existing systems or DevOps pipelines can require extra effort or may not always be straightforward.
- **Limited Model Types:** Certain advanced machine learning models (e.g., reinforcement learning) may not be well-supported, limiting its use for highly specialized applications.

Conclusion:

Edge Impulse is a powerful tool for simplifying machine learning on edge devices, especially for developers working with IoT or embedded systems. It excels in ease of use, optimization for resource-constrained environments, and hardware support. However, its limitations in customization, dependency on the cloud, and challenges with complex models may make it less suitable for highly specialized or large-scale projects.

Chapter 4

Problem Analysis

4.1 Overview

The convergence of artificial intelligence (AI) and motion tracking technologies represents a pivotal moment for innovation across various fields, including healthcare, sports analytics, robotics, and entertainment. Despite their potential, the integration of these two domains faces numerous challenges, which hinder accessibility and scalability. This chapter delves into the problem space by analyzing the current limitations, defining the research objectives, and establishing the scope and significance of the study.

By addressing these challenges, the proposed research seeks to contribute to the democratization of AI-driven motion tracking systems, fostering widespread adoption and innovation.

4.2 Background and Context

Motion tracking, enabled by devices such as inertial measurement units (IMUs), cameras, and wearable sensors, is a critical enabler of applications that require precise understanding of movement. Recent advancements have led to the development of systems capable of tracking complex motions in real-time. Examples include:

- Healthcare: Monitoring rehabilitation progress and detecting fall risks.
- Sports: Analyzing athletes' performance to optimize training.
- Entertainment: Enabling realistic motion capture in animation and gaming.

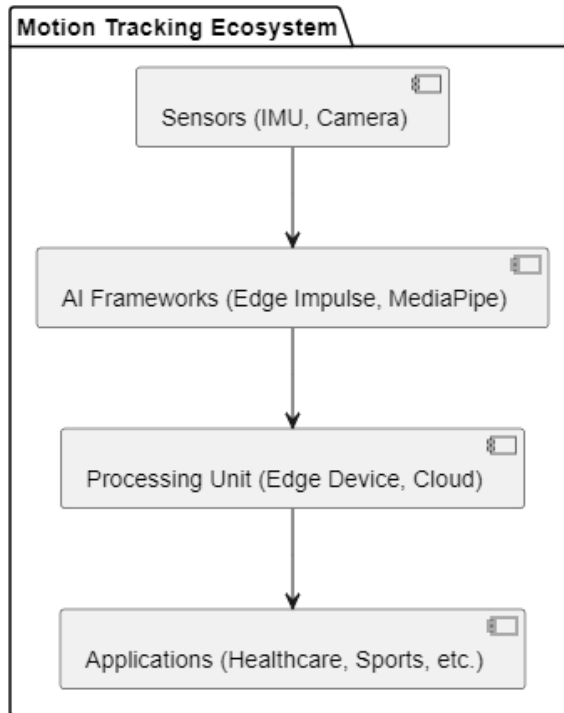


Figure 4.2.1: Motion Tracking Ecosystem Diagram

However, integrating motion tracking with AI introduces complexities that stem from technological and practical barriers. Although platforms such as Edge Impulse and Google MediaPipe offer powerful capabilities, their utility is often constrained by:

- The need for technical expertise to design, train, and deploy AI models.
- Hardware limitations that restrict real-time processing on edge devices.
- A lack of standardization across datasets, making it challenging to generalize AI models.

Without a user-friendly framework, these challenges remain significant hurdles for researchers, developers, and end-users alike.

4.3 Problem Definition

Despite the potential of AI to revolutionize motion tracking, several key issues persist:

1. **Complexity for Non-Experts:** The creation and deployment of AI models for motion tracking often require expertise in multiple domains, including machine learning, sensor fusion, and hardware optimization.

For instance, researchers must preprocess raw sensor data, extract meaningful features, and train models, all of which demand specialized knowledge.

2. **Scalability and Adaptability:** Many existing solutions are tailored for specific use cases, such as clinical gait analysis or virtual reality applications, making them unsuitable for broader adoption.

Lack of modularity in frameworks hinders adaptability to diverse scenarios.

3. **Hardware Limitations:** Resource-constrained devices like wearables have limited computational power, memory, and battery life, making it difficult to implement and run complex AI models.

4. **Data Challenges:** Motion tracking systems depend heavily on high-quality data, yet variability in sensor performance, environmental factors, and user movements often lead to noisy or incomplete datasets.

5. **Cost Barriers:** Advanced motion tracking systems, such as optical motion capture setups, are prohibitively expensive, restricting their use to high-budget projects or well-funded institutions.

Table 4.3.1: Problem Summary Table

Challenge	Description	Impact
Complexity	Requires expertise in AI, sensor fusion, and hardware optimization.	Limited accessibility for non-experts.
Scalability	Frameworks often tailored to specific use cases.	Difficult to generalize or adapt.
Hardware Limitations	Limited computational power on edge devices.	Performance bottlenecks.
Data Challenges	Variability and noise in motion data.	Reduces model accuracy.
Cost Barriers	Advanced systems are expensive.	Limits adoption in low-budget settings.

Addressing these challenges necessitates a framework that simplifies AI model generation while being resource-efficient, adaptable, and accessible.

4.4 Objectives of the Research

This research aims to overcome the identified challenges by focusing on the following objectives:

1. **Framework Development:** Develop a user-friendly framework that facilitates the creation and deployment of AI models for motion tracking. The framework should abstract the technical complexities to enable usage by non-experts.
2. **Compatibility with Edge Devices:** Design the framework to be compatible with resource-constrained devices, such as the Seeed XIAO nRF52840 Sense, ensuring real-time processing with minimal energy consumption.
3. **Data Preprocessing and Model Optimization:** Incorporate automated methods for data preprocessing, feature extraction, and model optimization to improve accuracy and robustness.
4. **Evaluation and Comparison:** Provide tools for evaluating and comparing AI models across various motion tracking applications, enabling users to choose the most suitable model for their needs.

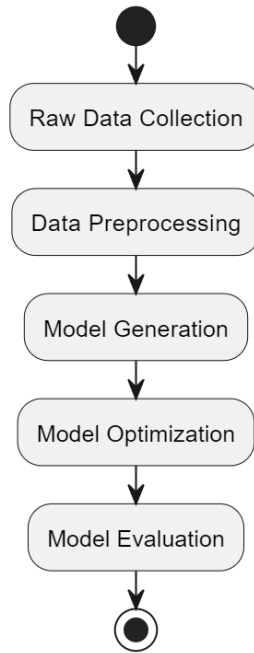


Figure 4.4.1: Framework Goals Flowchart

5. Scalability: Ensure the framework supports diverse applications, from healthcare monitoring to sports analytics, while being modular and adaptable.

4.5 Scope of the Research

The research is focused on developing a framework that bridges the gap between motion tracking systems and AI. Key aspects of the scope include:

- **Primary Focus:** The study will center on designing and implementing an AI model generation framework tailored for motion tracking applications.
- **Exclusions:** While the research explores motion data processing and AI integration, it will not delve into unrelated AI domains such as natural language processing or image-based tracking.
- **Hardware Agnosticism with Specific Examples:** Although the proposed framework will be evaluated using specific hardware like the Seeed XIAO nRF52840 Sense, the design principles will be adaptable to other devices.

This scoped approach ensures the research remains focused and achievable within the given timeframe.

4.6 Significance of the Study

The proposed framework addresses several key gaps in current motion tracking and AI systems:

1. For Researchers: Provides a structured approach to model development, reducing time and effort spent on repetitive tasks. Promotes innovation by enabling experiments with different AI models and techniques.
2. For Developers: Reduces the technical barriers to entry, fostering broader adoption of AI in motion tracking applications.
3. For End Users: Enables cost-effective solutions that can be deployed in real-world scenarios, such as wearable health monitoring or virtual reality gaming.

Table 4.6.1: Research Question Mapping

Research Question	Related Objective	Challenge Addressed
How can the framework abstract complexities?	Develop a user-friendly framework.	Complexity, Scalability.
What preprocessing techniques are most effective?	Automate preprocessing and feature extraction.	Data Challenges.
How can the framework support edge devices?	Optimize for resource-constrained devices.	Hardware Limitations.
What metrics best evaluate AI motion tracking systems?	Provide robust evaluation methods.	Scalability, Usability.

By simplifying AI model generation and deployment, the framework could accelerate the adoption of AI-powered motion tracking systems, benefiting multiple industries and stakeholders.

4.7 Research Questions

The following research questions guide the study:

1. How can the proposed framework abstract the complexities of AI model generation for motion tracking applications?
2. What preprocessing and optimization techniques are most effective for motion data?
3. How can the framework be adapted to resource-constrained devices without compromising performance?
4. What evaluation metrics best reflect the performance of AI models in motion tracking systems?

4.8 Summary

This chapter provided a comprehensive analysis of the research problem, defined the objectives, and outlined the scope and significance of the study. By addressing

the challenges identified, the research aims to make AI-powered motion tracking systems more accessible, scalable, and effective. The next chapter will detail the research methodology, laying the groundwork for achieving these objectives.

Chapter 5

Methodology

5.1 Overview

The proposed framework this study is designed to develop robust and efficient AI models for motion tracking applications, focusing on practical implementation, scalability, and ease of use. Combining theoretical exploration, empirical validation, and modular design, the framework addresses key challenges such as data quality, computational constraints, and user accessibility. By integrating data preprocessing, model development, and a user-friendly interface, it aims to deliver a comprehensive and effective solution for motion tracking systems.

The framework is structured around three key components:

- **Data Collection and Preprocessing**

Motion data collected from various sensors, such as accelerometers and gyroscopes, often produces noisy and inconsistent outputs. To ensure reliable data and model training readiness, the framework uses robust preprocessing techniques, including filtering, normalization, and data augmentation. These methods help reduce noise, handle missing values, and enrich the dataset, improving the generalizability of models.

- **AI Model Development and Optimization**

The framework emphasizes the creation of lightweight and efficient AI architectures specifically designed for resource-constrained environments such as edge devices. The models are designed to achieve high accuracy and real-time performance while maintaining computational efficiency. The iterative development

process ensures continuous refinement, adapting models to different motion patterns and tracking requirements.

- **System Integration and User Interface**

Seamless integration across multiple hardware and software environments is a core focus of the framework. Additionally, a user-friendly interface allows users to visualize motion data, interpret model output, and customize system settings with ease. This design prioritizes accessibility, allowing both expert and non-expert users to use the system effectively.

KEY CHALLENGES

Despite its well-structured design, the framework must address several challenges to ensure success:

- **Data quality and variability**

Motion sensor data often contains noise, inconsistencies, and missing values, which can hinder model performance. Advanced preprocessing techniques are required to normalize and prepare the data for training.

- **Integration and usability**

Ensuring compatibility with a variety of hardware setups and maintaining a user-friendly experience for non-experts requires careful design and testing.

- **Privacy and ethical concerns**

The collection of motion data raises privacy concerns. The framework integrates data anonymization methods and compliance with data protection regulations to ensure the ethical use of data.

POTENTIAL IMPACTS

The proposed framework has the potential to bring transformative benefits to the motion tracking field and its applications:

- **Advanced motion tracking technology**

By providing a robust, scalable, and efficient solution, the framework can significantly improve the accuracy and reliability of motion tracking systems, enabling detailed analysis of human movements.

- **Enabling industry applications**

From healthcare (e.g., rehabilitation and physical therapy) to fitness (e.g., activity tracking) and entertainment (e.g., motion capture in games and movies), the framework enables versatile applications across multiple domains.

- **Democratizing Accessibility**

A focus on user-friendly design ensures that the technology is accessible to a wider audience, including researchers, developers, and practitioners with limited technical expertise.

By addressing these challenges and emphasizing innovation, scalability, and usability, the proposed framework aims to set a new standard for motion tracking systems. Its potential to revolutionize motion tracking technology opens up exciting possibilities for advancing research and industry applications.

5.2 Research Design

The research design outlines the high-level structure of the study, defining how the objectives will be translated into actionable steps. It emphasizes a systematic, iterative approach to ensure flexibility and adaptability during the research process. The design is divided into the following stages:

5.2.1 Literature Review

The first stage involves an in-depth examination of existing literature in motion tracking systems, AI model generation frameworks, and edge computing technologies. Key objectives include:

- Identifying current advancements in motion tracking and their application areas (e.g., healthcare, sports, and human-computer interaction).
- Analyzing the capabilities and limitations of frameworks like Edge Impulse and Google MediaPipe.
- Understanding the challenges posed by motion data, such as noise, heterogeneity, and the need for accurate annotation.

This stage sets the foundation for identifying gaps and opportunities that inform the design of the proposed framework.

5.2.2 Requirement Analysis

Requirement analysis bridges the gap between theoretical insights and practical implementation. Key tasks include:

Functional Requirements:

- Support for multiple input sources, including IMU sensors and external datasets.
- Capability to automate data preprocessing, feature extraction, and model training.
- Seamless integration with edge devices and mobile applications.

Non-Functional Requirements:

- Scalability to handle increasing data volumes or new sensor modalities.
- Resource efficiency, ensuring models are lightweight and computationally feasible for edge devices.
- User-friendliness to cater to both technical and non-technical users.

These requirements serve as benchmarks during the framework development and evaluation stages.

5.2.3 Framework Development

The framework is developed iteratively to incorporate user feedback and address technical challenges. This process involves:

- Designing a modular architecture that separates preprocessing, model generation, and deployment components.
- Leveraging existing frameworks, such as TensorFlow Lite, for rapid prototyping while ensuring the flexibility to integrate custom algorithms.
- Incorporating a graphical user interface (GUI) to simplify user interactions.

5.2.4 Experimental Validation

Validation ensures that the framework meets its functional and performance requirements. Experiments are conducted to evaluate the framework under diverse conditions:

- Data Variability: Test with various motion types and intensities.

- **Hardware Constraints:** Assess performance on edge devices like the Seeed XIAO nRF52840 Sense.
- **User Scenarios:** Simulate real-world applications, such as fitness tracking or gesture recognition.

5.2.5 Evaluation and Iteration

Evaluation is an ongoing process, with feedback loops integrated at every stage. Metrics such as accuracy, latency, and user satisfaction guide refinements to the framework. Multiple iterations are conducted to ensure robustness and usability.

5.3 Tools and Technologies

The choice of tools and technologies is critical for addressing the unique requirements of motion tracking applications. A combination of hardware, software, and development platforms is used to create and validate the framework.

5.3.1 Hardware

- **Seeed XIAO nRF52840 Sense:** This microcontroller serves as the primary sensor hub, equipped with an onboard IMU to capture motion data and BLE capabilities for wireless communication. Its compact size and low power consumption make it ideal for edge applications.
- **BLE-Enabled Smartphones:** Smartphones are used to receive data transmitted by the Seeed XIAO and provide a user-friendly interface for data visualization and analysis.

5.3.2 Software and Libraries

- **Edge Impulse:** A no-code AI development platform, Edge Impulse enables rapid prototyping of AI models for edge devices. Its preprocessing and feature extraction pipelines are particularly relevant for motion data.
- **Google MediaPipe:** MediaPipe's gesture recognition pipelines offer insights into designing efficient, real-time motion tracking models.
- **TensorFlow Lite:** TensorFlow Lite allows for the deployment of lightweight AI models on edge devices, optimizing for speed and memory efficiency.

- Arduino IDE and PlatformIO: These platforms are used to program the Seeed XIAO and develop firmware for sensor data collection and transmission.

5.3.3 Development Environment

A hybrid environment is adopted, combining:

- Local Tools: Python scripts for preprocessing and model training.
- Cloud Platforms: Edge Impulse Studio for data preprocessing and AI model benchmarking.

5.4 Data Collection and Preprocessing

5.4.1 Data Sources

Primary Data: Raw motion data collected using the IMU sensors on the Seeed XIAO nRF52840 Sense. Supplementary Data: Synthetic datasets generated to complement real-world data, ensuring diverse motion scenarios.

5.4.2 Preprocessing Steps

- Noise Reduction: Filtering techniques, such as low-pass or Kalman filters, are applied to mitigate sensor noise.
- Segmentation: Motion data is divided into fixed-size windows, enabling the identification of discrete activities.
- Feature Extraction: Key features are extracted from each segment, including:
 - Statistical metrics (e.g., mean, variance).
 - Frequency-domain features (e.g., spectral energy).
 - Derived metrics like acceleration magnitude and orientation angles.
- Data Normalization: All features are scaled to a uniform range for consistency during model training.

5.5 Framework Development

5.5.1 Architecture

The framework is designed to operate as an end-to-end system, consisting of:

- Input Module: Handles raw data ingestion from IMU sensors.
- Processing Module: Automates noise filtering, segmentation, and feature extraction.
- Modeling Module: Supports training of AI models using predefined templates or custom algorithms.
- Output Module: Generates optimized models for deployment on edge devices.

5.5.2 Workflow

1. Data is collected from the Seeed XIAO and ingested into the framework.
2. Preprocessing pipelines clean and prepare the data.
3. AI models are trained, evaluated, and exported for deployment.

5.6 Ethical Considerations

Ethical issues are considered throughout the research, ensuring:

- Data Privacy: Safeguards are in place to anonymize motion data.
- Inclusivity: Diverse datasets are collected to avoid algorithmic biases.

5.7 Summary

This chapter provided a comprehensive methodology for developing, implementing, and evaluating the proposed framework. Subsequent chapters will delve into the experimental setup, results, and analysis.